Connect MySQL to Django

Complete step-by-step guide for connecting a Django project to MySQL 8.4.

1. Prerequisites

You should have Python, Django, a Django project, MySQL 8.4 or later, and a working Python virtual environment.

Activate your virtual environment, for example:

```
[redacted].virtualenvs\Learn-django-UbJiBmgc\Scripts\activate
```

2. Check the MySQL version

```
mysql --version
```

Django versions that require MySQL 8.4 will reject MySQL 8.0.x. If you see 8.0.46, upgrade to MySQL 8.4 before continuing.

3. If upgrading MySQL, back up your databases first

The mysqldump command must be executed from a normal Windows terminal, not from the mysql> prompt.

Exit the MySQL prompt:

```
exit;
```

Create a backup folder and export all databases:

```
mkdir C:\mysql_backup
mysqldump -u root -p --all-databases > C:\mysql_backup\backup.sql
dir C:\mysql_backup
```

If you previously typed mysqldump inside mysql>, that is why you may have received errors such as Unknown command '\m' or '\b'.

4. Upgrade MySQL to 8.4

After confirming that the backup.sql file exists, stop the old MySQL service and uninstall MySQL 8.0 through Windows Settings → Apps → Installed apps. Do not manually delete your MySQL data directory before confirming your backup.

```
net stop MySQL80
```

Install MySQL 8.4, then open a new terminal and check:

```
mysql --version
```

If the command still reports 8.0.46, check which executable Windows is using:

```
where mysql
```

Make sure the MySQL 8.4 bin directory is in PATH, commonly:

```
C:\Program Files\MySQL\MySQL Server 8.4\bin
```

5. Start the MySQL service

The service name may be MySQL84 or another name chosen during installation.

```
net start MySQL84
```

If that service name does not work, find the installed MySQL service:

```
sc query type= service | findstr /I MySQL
```

6. Restore the backup

Run the restore from CMD, PowerShell, or Git Bash—not from the mysql> prompt:

```
mysql -u root -p < C:\mysql_backup\backup.sql
```

Then verify the databases:

```
mysql -u root -p
SHOW DATABASES;
SELECT VERSION();
exit;
```

Because the backup was made with --all-databases, it should include django_db if that database existed when the backup was created.

7. Create the Django database if it does not already exist

Only do this if SHOW DATABASES does not already show django_db.

```
mysql -u root -p
CREATE DATABASE django_db;
SHOW DATABASES;
exit;
```

8. Install the MySQL Python driver

```
pip install mysqlclient
```

If mysqlclient fails to install on Windows, keep the exact pip error; it can indicate a missing build dependency or an incompatible Python package.

9. Configure Django settings.py

Open your project's settings.py and replace the SQLite DATABASES configuration with:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'django_db',
        'USER': 'root',
        'PASSWORD': 'YOUR_MYSQL_PASSWORD',
        'HOST': 'localhost',
        'PORT': '3306',
        'OPTIONS': {
            'init_command': "SET sql_mode='STRICT_TRANS_TABLES'",
        },
    }
}
```

Do not commit a real database password to GitHub. For a real project, use environment variables or a secrets manager. Since a password was previously exposed in the conversation, change that password before using it in a real environment.

10. Test the Django/MySQL connection

```
python manage.py check
```

If this completes successfully, Django can load the project configuration. Next, create and apply migrations.

11. Create and apply migrations

Generate migration files:

```
python manage.py makemigrations
```

Apply the migrations to MySQL:

```
python manage.py migrate
```

Remember: makemigrations creates migration files from model changes; migrate applies those migrations to the database.

12. Verify the Django tables in MySQL

```
mysql -u root -p
USE django_db;
SHOW TABLES;
exit;
```

You should see Django tables such as django_migrations and other application tables after migrations have been applied.

13. Create a Django admin user

```
python manage.py createsuperuser
```

14. Start the Django development server

```
python manage.py runserver
```

Then open http://127.0.0.1:8000/ in your browser. The Django admin is normally available at /admin/.

15. Normal Django/MySQL workflow

```
python manage.py check
python manage.py makemigrations
python manage.py migrate
python manage.py runserver
```

16. Understanding the important commands

```
mysql --version
    → Shows the installed MySQL client version.

mysql -u root -p
    → Opens the MySQL command-line client.

mysqldump -u root -p --all-databases > backup.sql
    → Creates a SQL backup.

mysql -u root -p < backup.sql
    → Restores a SQL backup.

python manage.py check
    → Checks the Django project configuration.

python manage.py makemigrations
    → Creates migration files from model changes.

python manage.py migrate
    → Applies migrations to the database.

python manage.py createsuperuser
    → Creates a Django admin account.

python manage.py runserver
    → Starts the Django development server.
```

17. Troubleshooting

Error: 'mysql' is not recognized as an internal or external command

MySQL is probably installed but its bin directory is not in PATH. Use where mysql and add the correct MySQL 8.4 bin directory to PATH, then reopen the terminal.

Error: MySQL 8.4 or later is required (found 8.0.46)

Your Django version requires MySQL 8.4 or later. Upgrade MySQL and verify with mysql --version. If the old version still appears, use where mysql to locate the old executable.

Error: Unknown command '\m' or '\b' while entering mysqldump

You ran mysqldump inside the MySQL client. Exit with exit; and run mysqldump from CMD, PowerShell, or Git Bash.

Error while running makemigrations

First run python manage.py check and inspect the exact traceback. If the error says MySQL 8.4 is required, verify the MySQL version and PATH before changing your Django models.

18. Final checklist

```
[ ] MySQL 8.4 or later is installed
[ ] mysql --version shows 8.4.x
[ ] MySQL service is running
[ ] django_db exists (or was restored from backup)
[ ] mysqlclient is installed in the active virtual environment
[ ] settings.py uses django.db.backends.mysql
[ ] Database name, user, password, host and port are correct
[ ] python manage.py check succeeds
[ ] python manage.py makemigrations succeeds
[ ] python manage.py migrate succeeds
[ ] SHOW TABLES; shows Django tables
[ ] python manage.py createsuperuser succeeds
[ ] python manage.py runserver starts the application
```